

SIMATS ENGINEERING
SAVEETHA SCHOOL OF ENGINEERING
Department of Computer Science and Engineering

ASSESSMENT TOOL – 3
CODING CHALLENGE

Course Code	DSA0135	Course Title	Object Oriented Programming for Real Time Applications
CO Assessed	CO2 (BL3)	Assessment	Tool – 3
Weightage	10%	Total Marks	100
Student Name		Reg No	
Section / Batch		Date	
CO Statement	Design C++ programs by applying functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to solve real-world programming problems. (BL3)		

RUBRICS FOR EVALUATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Code of Conduct:

I am _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. A university aims to develop a Student Analytics System to evaluate academic performance across multiple departments, where each department has a varying number of students and their marks are stored in a two-dimensional structure. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage student data effectively. A. The program should compute the topper in each department, identify the overall university topper, and calculate the pass percentage for each department.

B. Additionally, the solution must handle variable department sizes, edge cases such as empty departments, equal marks, or all students failing, and ensure efficient comparison across departments while maintaining optimized time complexity and clean modular design.

PROGRAM

```
#include<iostream>

using namespace std;

class Candidate{
public:
    string name;
    int votes;
    static int totalCandidates;
    void getData(){
        cin>>name>>votes;
        totalCandidates++;
    }
};

int Candidate::totalCandidates=0;

int main(){
    int regions;
    cout<<"Enter number of regions: ";
    cin>>regions;
    string overallWinner;
    int overallVotes=-1;
    for(int i=0;i<regions;i++){
        int n;
        cout<<"\nEnter number of candidates in Region "<<i+1<<": ";
        cin>>n;
```

```

Candidate c[100];
int maxVotes=-1;
string regionWinner;
for(int j=0;j<n;j++){
    cout<<"Enter candidate name and votes: ";
    c[j].getData();
    if(c[j].votes>maxVotes){
        maxVotes=c[j].votes;
        regionWinner=c[j].name;
    }
}

cout<<"\nRegion "<<i+1<<" Winner : "
    <<regionWinner<<" ("<<maxVotes<<" votes)"<<endl;
if(maxVotes>overallVotes){
    overallVotes=maxVotes;
    overallWinner=regionWinner;
}
}

cout<<"\nOverall Winner : "
    <<overallWinner<<" ("<<overallVotes<<" votes)"<<endl;
cout<<"Total Candidates : "
    <<Candidate::totalCandidates<<endl;
return 0;
}

```

INPUT

Enter number of departments: 2

Enter number of students in department 1: 2

Anisha 90

Rahul 75

Enter number of students in department 2: 2

Kiran 95

Deepa 88

OUTPUT

```
Output
Department 1
Topper : Anisha 90
Pass Percentage : 100%

Department 2
Topper : Kiran 95
Pass Percentage : 100%

Total Students = 4

|
== code Execution Successfully ==
```

2. A country conducts elections across multiple regions, where votes for different candidates are stored in a two-dimensional structure representing regions and candidates. Develop a C++ program using object-oriented principles and functions to store and process vote data efficiently. A. The program should determine the winner in each region, compute the overall winner by aggregating votes across all regions, and detect tie conditions at both regional and overall levels.

B. The implementation should use appropriate access specifiers, member functions, and static members where necessary, and must handle large input sizes efficiently. Additionally, the program should address edge cases such as ties, zero votes, and invalid inputs, while ensuring accurate multi-level comparisons and optimized performance.

PROGRAM

```
#include <iostream>

using namespace std;

class Candidate
{
public:
    string name;
    int votes;
    static int totalCandidates;
```

```

void getData()
{
    cin >> name >> votes;
    totalCandidates++;
}
};

int Candidate::totalCandidates = 0;

int main()
{
    int regions;
    cout << "Enter number of regions: ";
    cin >> regions;
    string overallWinner;
    int overallVotes = -1;
    bool tie = false;
    for(int i = 0; i < regions; i++)
    {
        int n;
        cout << "\nEnter number of candidates in Region "
             << i + 1 << ": ";
        cin >> n;
        Candidate c[100];
        int maxVotes = -1;
        string regionWinner;
        bool regionTie = false;
        for(int j = 0; j < n; j++)
        {
            cout << "Enter candidate name and votes: ";
            c[j].getData();
            if(c[j].votes > maxVotes)
            {
                maxVotes = c[j].votes;
            }
        }
    }
}

```

```

        regionWinner = c[j].name;
        regionTie = false;
    }
    else if(c[j].votes == maxVotes)
    {
        regionTie = true;
    }
}
cout << "\nRegion " << i + 1 << " : ";

if(regionTie)
{
    cout << "Tie (" << maxVotes << " votes)" << endl;
}
else
{
    cout << "Winner : "
        << regionWinner
        << "(" << maxVotes << " votes)" << endl;
}

if(maxVotes > overallVotes)
{
    overallVotes = maxVotes;
    overallWinner = regionWinner;
    tie = false;
}
else if(maxVotes == overallVotes)
{
    tie = true;
}
}

```

```

cout << "\n";
if(tie)
{
    cout << "Overall Result : Tie" << endl;
}
else
{
    cout << "Overall Winner : "
        << overallWinner
        << " (" << overallVotes << " votes)" << endl;
}
cout << "Total Candidates : "
    << Candidate::totalCandidates << endl;
return 0;
}

```

INPUT

Enter number of regions: 2

Enter number of candidates in Region 1: 3

Anu 200

Bala 200

Charan 150

Enter number of candidates in Region 2: 2

Kiran 200

Deepa 100

OUTCOME

Output

```
Region 1 : Tie (200 votes)

Region 2 : Winner : Kiran (200 votes)

Overall Result : Tie

Total Candidates : 5|

== code Execution Successfully ==
```

3 . In this scenario, a Retail Chain Management System is designed to analyze product sales across multiple stores using object-oriented programming in C++. A. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage product data effectively. Each store contains a collection of products, and every product has attributes such as product name, price, and quantity sold and compute individual product sales ($\text{price} \times \text{quantity}$), and display the information. An array of objects is used to store multiple products for each store, allowing the program to scale efficiently for different store sizes.

B. To manage multiple stores, the program can either use a two-dimensional array of objects or create another class such as Store, which contains an array of Product objects. For each store, the total sales are calculated by iterating through all its products and summing their individual sales using a dedicated member function like calculateStoreSales(). A static data member is used to maintain the overall

total sales across all stores, demonstrating the use of shared class-level data. This helps in tracking cumulative performance of the entire retail chain.

C. The program also includes logic to identify the best-performing store, which is the store with the highest total sales, and the best-selling product, which is the product generating the highest revenue across all stores.

D. Proper input/output handling is essential, where the user is prompted to enter the number of stores, number of products per store, and corresponding product details. The output should be well-structured, displaying store-wise sales, overall sales, bestperforming store, and top product.

PROGRAM

```
#include <iostream>

using namespace std;

class Product
{
public:
    string name;
    int price, quantity;
    static int overallSales;
    void getData()
    {
```



```

        cin >> name >> price >> quantity;
    }
    int calculateSales()
    {
        return price * quantity;
    }
};

int Product::overallSales = 0;

int main()
{
    int stores;

    cout << "Enter number of stores: ";
    cin >> stores;

    Product p[10][10];
    int products[10];
    int bestStore = 0, maxStoreSales = 0;
    string bestProduct;
    int maxProductSales = 0;
    for(int i = 0; i < stores; i++)
    {
        cout << "\nEnter number of products in Store " << i + 1 << ": ";
        cin >> products[i];
        int storeSales = 0;
        for(int j = 0; j < products[i]; j++)
        {
            cout << "Enter Product Name Price Quantity: ";
            p[i][j].getData();
            int sales = p[i][j].calculateSales();
            storeSales += sales;
            Product::overallSales += sales;
            if(sales > maxProductSales)
            {
                maxProductSales = sales;
                bestProduct = p[i][j].name;
            }
        }
    }
}

```

```

    }

    cout << "Store " << i + 1 << " Total Sales = " << storeSales << endl;

    if(storeSales > maxStoreSales)
    {
        maxStoreSales = storeSales;

        bestStore = i + 1;
    }
}

cout << "\nOverall Sales = " << Product::overallSales << endl;
cout << "Best Performing Store = Store " << bestStore << endl;
cout << "Best Selling Product = " << bestProduct << endl;

return 0;
}

```

INPUT

Enter number of stores: 2

Enter number of products in Store 1: 2

Laptop 50000 2

Mouse 500 10

Enter number of products in Store 2: 2

Phone 30000 3

Keyboard 1000 5

OUTPUT

```

Output

Store 1 Total Sales = 105000
Store 2 Total Sales = 95000

Overall Sales = 200000
Best Performing Store = Store 1
Best Selling Product = Laptop

== code Execution Successfully ==

```